

ML для старта в Data Science

## **5. Методы опорных векторов (SVM)**

Жарова Мария

# План занятия, теория + практика

- Разбор SVM и его модификаций
- Ещё метрики классификации
- Практика: классификация с логрег. и SVM

*Пишите вопросы в чат в любое время!  
Если писать много – обсудим голосом в  
специальных перерывах :)*

# Разбор SVM и его модификаций

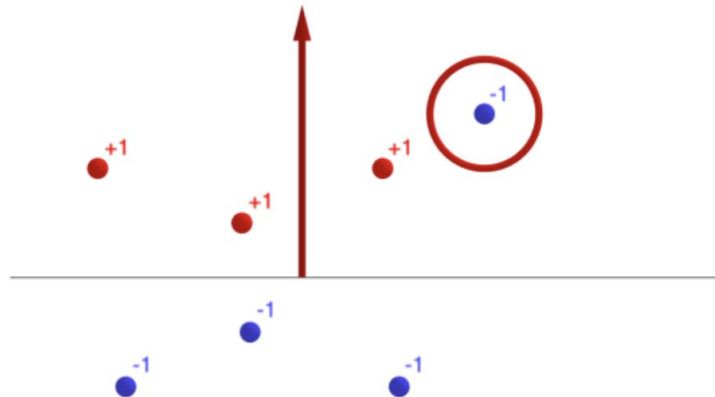
# Ресар: отступ (margin)

– это мера того, насколько уверена модель в своем прогнозе для конкретного объекта.

***Больше отступ == больше уверенность!***

$$M(x_i, y_i) = y_i * x_i \cdot w,$$

здесь  $y_i$  – метка класса (-1 или +1).



# Ресар: геометрическая интерпретация

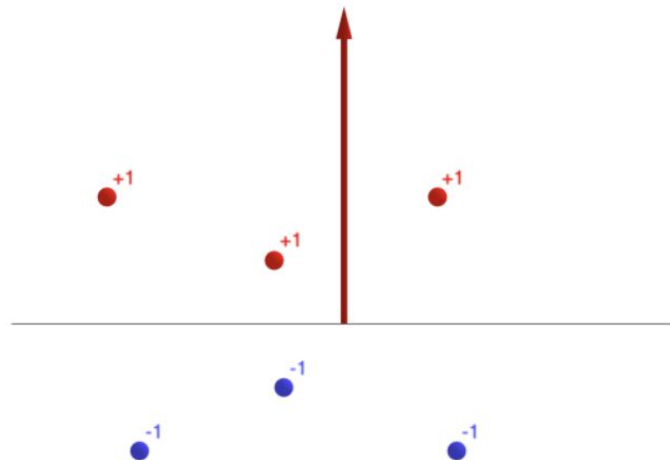
Формализуем понятие вектора нормали:

$$w = (w_1, w_2, \dots, w_d),$$

где  $d$  — число признаков,

$w_1, w_2, \dots, w_d$  — коэффициенты модели.

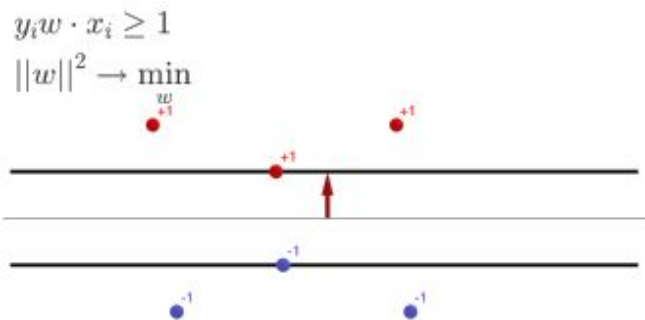
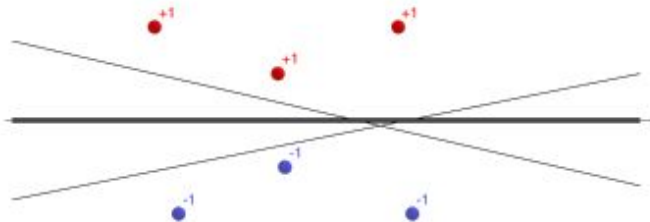
Если  $x_i \cdot w \geq 0$  — объект относится к положительному классу; иначе — к отрицательному.



# Алгоритм SVM

**SVM (Support vector Machine)** – алгоритм классификации, основывается на *поиске гиперплоскости в пространстве признаков, которая наилучшим образом разделяет данные на два класса.*

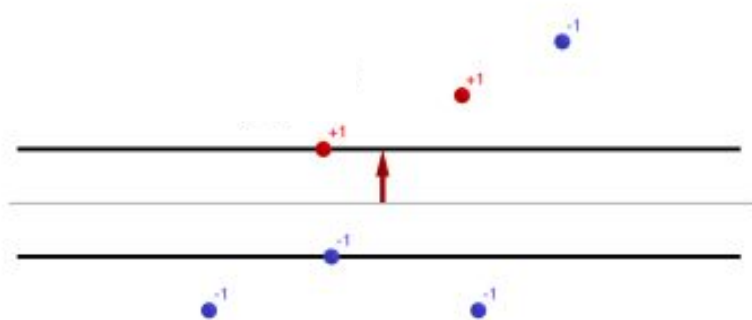
В случае SVM ищем **гиперплоскость, которая максимизирует зазор** (отступ между двумя классами данных).



# Алгоритм SVM: Hard Margin #1

**Зазор == опорный вектор** – в контексте SVM это расстояние от гиперплоскости до ближайшей точки каждого класса.

Если использовать **безусловную оптимизацию** – алгоритм не сработает на данных с выбросами.



PS: по умолчанию SVM – алгоритм для линейно separable выборок.

# Алгоритм SVM: Hard Margin #2

Математическая постановка задачи:

$$\begin{cases} \|w\|^2 \rightarrow \min_w (\|w\|^2 \text{ берется в квадрате для облегчения вычисления производных}) \\ |w \cdot x_i + w_0| \geq 1 \end{cases}$$

- максимизируем зазор – первое выражение,
- при условии, что ошибок нет (классы идеально линейно разделимы) – второе неравенство.

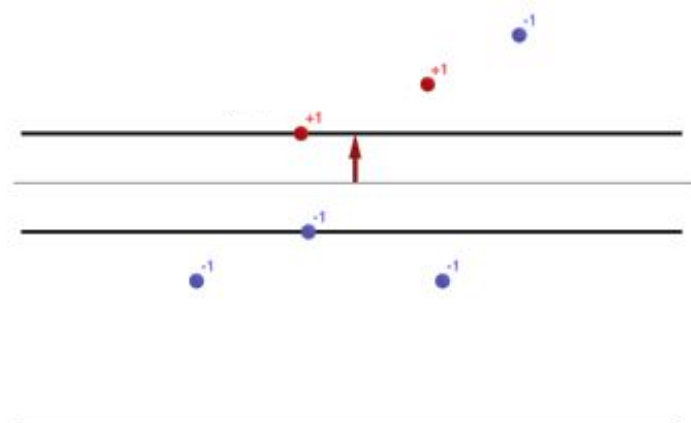


# Алгоритм SVM: Soft Margin #1

Для таких случаев вводятся дополнительные переменные  $\epsilon_i$ , которые разрешают на некоторых объектах делать отступ меньше единицы.

Также вводится коэффициент  $C$ , который позволяет выбирать, что важнее — сделать отступ шире либо точность модели лучше.

$$\begin{cases} y_i w \cdot x_i \geq 1 - \epsilon_i \\ \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \epsilon_i \rightarrow \min_w \end{cases}$$



# Алгоритм SVM: Soft Margin #2

Математическая постановка задачи:

$$\begin{cases} y_i w \cdot x_i \geq 1 - \epsilon_i \\ \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \epsilon_i \rightarrow \min_w \end{cases}$$

- $y_i = \{-1, 1\}$  – класс объекта,
- $x_i$  – вектор его признаков,
- $w$  – вектор весов,
- $\epsilon_i$  – дополнительные переменные, обозначающие степень нарушения ограничений на margin для каждого объекта,
- $C$  – гиперпараметр, определяющий вес ошибки классификации в оптимизационной задаче.

# Алгоритм SVM: Soft Margin, пример #1

Рассмотрим пример: есть точки в двумерном пространстве:

- Класс +1: (2,3), (3,3), (2,2).
- Класс -1: (0,1), (1,1), (0,0).

Цель — найти разделяющую прямую  $w \cdot x + b = 0$ , которая наилучшим образом разделяет точки классов +1 и -1 с максимальным отступом.

# Алгоритм SVM: Soft Margin, пример #2

Рассмотрим пример: есть точки в двумерном пространстве:

- Класс +1: (2,3), (3,3), (2,2).
- Класс -1: (0,1), (1,1), (0,0).

Цель — найти разделяющую прямую  $w \cdot x + b = 0$ , которая наилучшим образом разделяет точки классов +1 и -1 с максимальным отступом.

Подставим координаты точек в условия, чтобы задать систему ограничений для нашего уравнения прямой.

Для точек класса +1:

$$\begin{cases} 2w_1 + 3w_2 + b \geq 1, \\ 3w_1 + 3w_2 + b \geq 1, \\ 2w_1 + 2w_2 + b \geq 1. \end{cases}$$

# Алгоритм SVM: Soft Margin, пример #3

Для точек класса +1:

$$\begin{cases} 2w_1 + 3w_2 + b \geq 1, \\ 3w_1 + 3w_2 + b \geq 1, \\ 2w_1 + 2w_2 + b \geq 1. \end{cases}$$

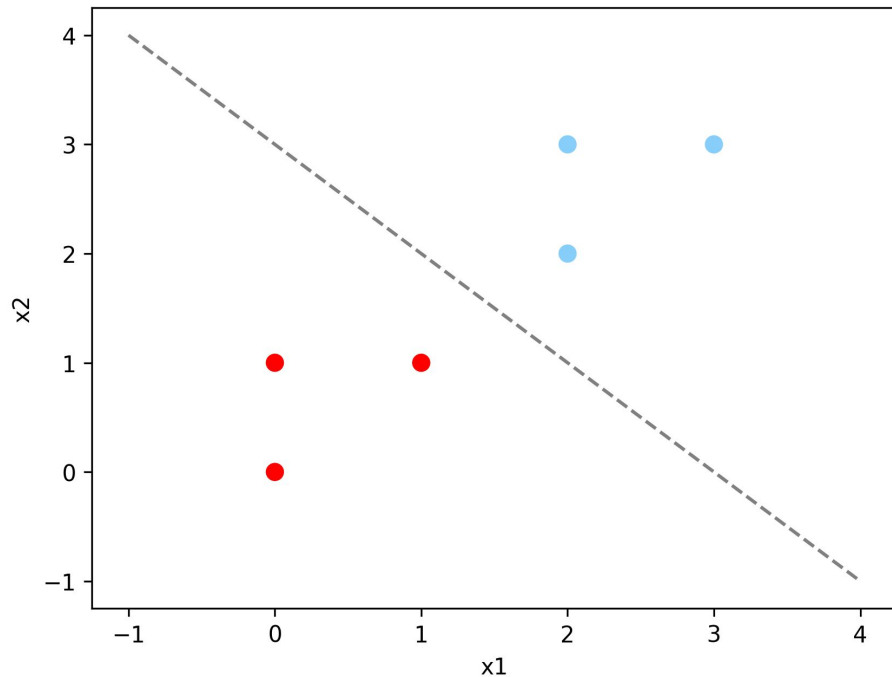
Для точек класса -1:

$$\begin{cases} b + w_2 \leq -1, \\ w_1 + w_2 + b \leq -1, \\ b \leq -1. \end{cases}$$

Эту задачу оптимизации можно решить численно (теми же градиентными методами, в Python – `scipy.optimize`).

В данном случае получим  $w_1 = 1$ ,  $w_2 = 1$ ,  $b = 3$ , т.е. следующее уравнение прямой:  $x_1 + x_2 + 3 = 0$

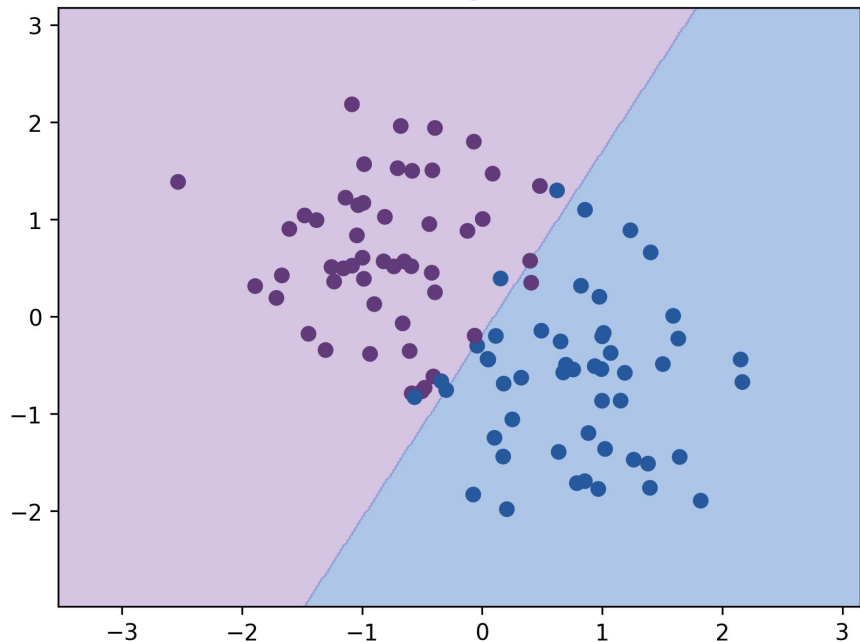
# Алгоритм SVM: Soft Margin, пример #4



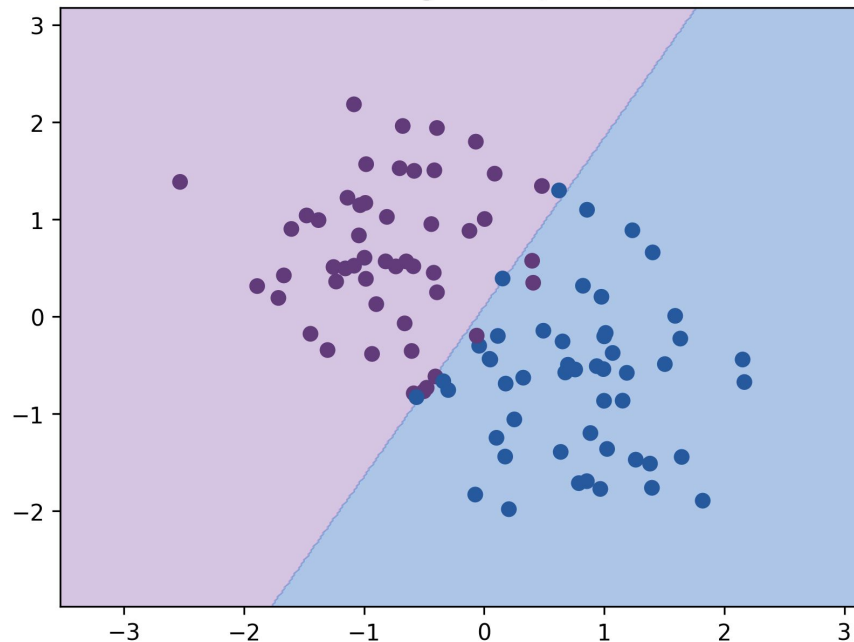
$$x_1 + x_2 + 3 = 0$$

# Hard Margin vs Soft Margin

Hard Margin SVM



Soft Margin SVM (C=1)



# Ресар: функция потерь Хинджа

**Функция потерь Хинджа** – максимизирует отступ между классами:

$$L(y, \hat{y}) = \max(0, 1 - y\hat{y}), \quad \text{или} \quad L(y_i, w) = \max(0, 1 - y_i(w^T x_i))$$

- 0 – если предсказание верно,
- иначе значение, пропорциональное расстоянию между предсказанием и истинной меткой класса.

Градиент этого лосса: 
$$\frac{\partial L(y_i, w)}{\partial w} = \begin{cases} -y_i \mathbf{x}_i, & w^T \mathbf{x}_i > 1 \\ 0, & \text{otherwise} \end{cases}$$

а далее можно запускать уже знакомый градиентный спуск:)



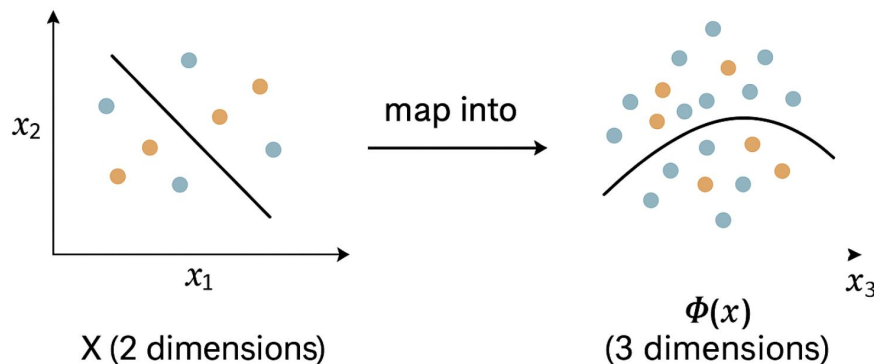
# Kernel Trick

SVM работает с скалярными произведениями в признаковом пространстве:

$$K(x, x') = \langle \phi(x), \phi(x') \rangle,$$

где  $\phi(x)$  — отображение данных в новое (обычно более высокоразмерное) пространство.

**Kernel trick:** мы не считаем  $\phi(x)$  явно, а вместо этого используем ядро  $K(x, x')$ , которое напрямую вычисляет скалярное произведение в этом пространстве.

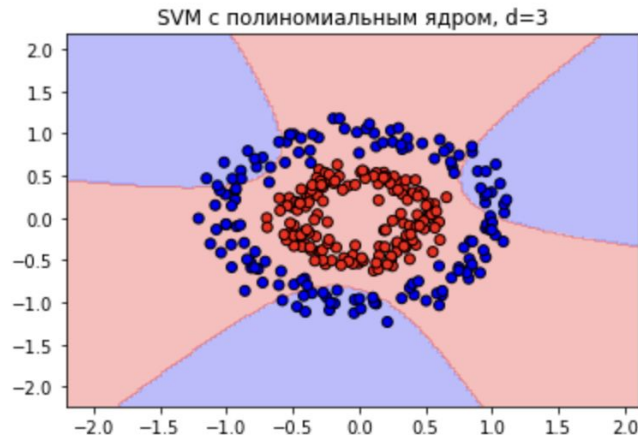
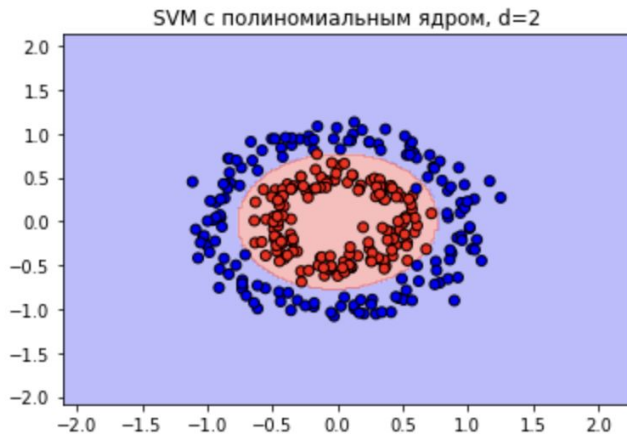


# Kernel Trick: полиномиальное ядро

Вид ядра следующий:  $K(x, x') = (\langle x, x' \rangle + c)^d$

здесь  $d$  — степень полинома (сколько «изгибов» границы допускаем),  
 $c$  — константа, регулирующая «жёсткость» разделителя.

**Идея:** позволяет моделировать границы как многочлены, полезно, если данные разделимы какой-то «кривой».



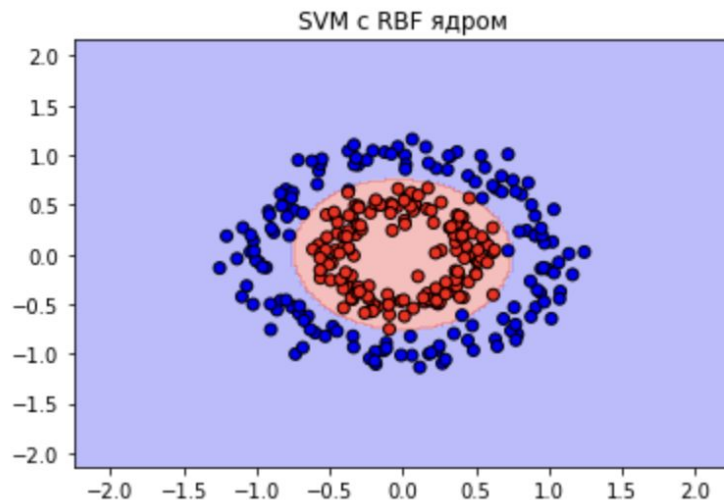
# Kernel Trick: RBF (Gaussian) ядро

Вид ядра следующий:  $K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$

$\sigma$  управляет шириной «колокола» (чем меньше, тем чувствительнее к локальным различиям).

- Очень гибкое, может выучить почти любую нелинейную границу.
- Но при слишком маленькой  $\sigma \rightarrow$  переобучение.

**Идея:** каждую точку окружает «аура», и модель учится по их пересечению.



# Шпаргалка по видам ядер

**Линейное ядро** — когда данные почти линейно разделимы или признаков очень много (тексты, bag-of-words).

**Полиномиальное ядро** — если предполагается, что граница имеет форму кривой, но не слишком сложной.

**RBF (Gaussian) ядро** — универсальный выбор, когда форма разделяющей границы неизвестна.

**Sigmoid ядро** — иногда применяют в нейросетевых задачах, но работает нестабильно.

**Laplacian ядро** — похоже на RBF, но более чувствительно к выбросам.

**Cosine ядро** — для данных, где важен угол между векторами, а не их длина (например, тексты).

# Важные детали

Масштабирование и удаление выбросов, как и в других линейных моделях, важно.

Веса также интерпретируются как степень влияния конкретного признака на таргет.

# Практика: классификация с логрег. и SVM

**Спасибо за внимание!**